

Shkenca Kompjuterike 2

Blerim Zylfiu - blerim.zylfiu@ubt-uni.net

Collections në Java



- Collection (Koleksioni) në Java është një framework (kornizë) që ofron një arkitekturë për të ruajtur dhe manipuluar një grup të objekteve.
- Collections në Java mund të arrijnë të gjitha operacionet që ju mund të kryeni me të dhëna (data) si : kërkimi, klasifikimi, futja e të dhënave, manipulimi dhe fshirja e tyre.
- Java Collection nënkupton një njësi të vetme të objekteve. Java Collection framework (kornizë) ofron shumë interface (Set, List, Queue, Deque) dhe klasa (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet etj).

Collections në Java

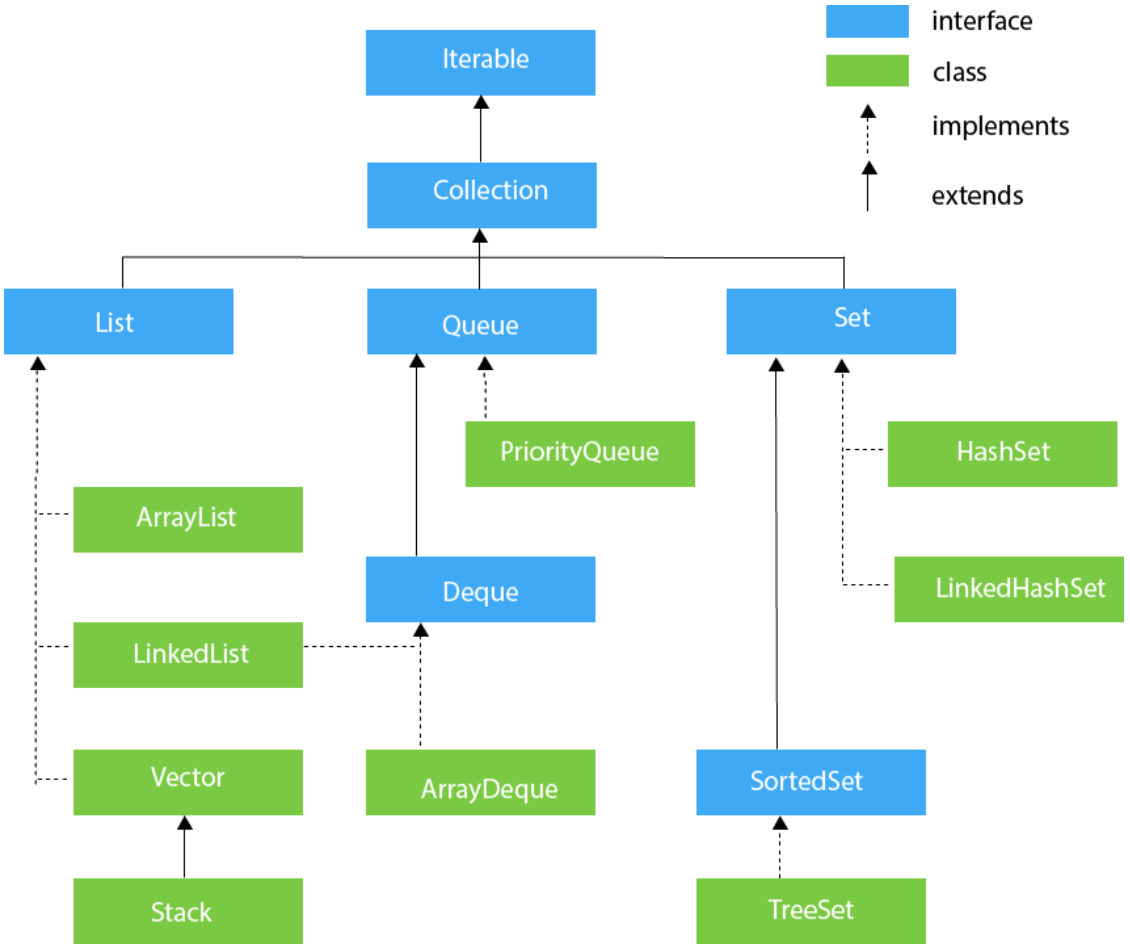
Çfarë është Collection në Java

- Një Collection përfaqëson një njësi të vetme të objekteve, d.m.th., një grup.

Çfarë është Collection framework

- Collection framework përfaqëson një arkitekturë të unifikuar për ruajtjen dhe manipulimin e një grupi objektesh. Ajo ka:
  - Interfaca dhe implementimet e tyre, d.m.th., klasat
  - Algoritmet

Hierarkia e Collection Framework



Disa metoda të Collection Interface



Method	Description
public boolean add(E e)	It is used to insert an element in this collection.
public boolean addAll(Collection<? extends E> c)	It is used to insert the specified collection elements in the invoking collection.
public boolean remove(Object element)	It is used to delete an element from the collection.
public boolean removeAll(Collection<?> c)	It is used to delete all the elements of the specified collection from the invoking collection.
public int size()	It returns the total number of elements in the collection.
public void clear()	It removes the total number of elements from the collection.
public boolean contains(Object element)	It is used to search an element.
public boolean containsAll(Collection<?> c)	It is used to search the specified collection in the collection.
public Iterator iterator()	It returns an iterator.
public Object[] toArray()	It converts collection into array.
public boolean isEmpty()	It checks if collection is empty.
public boolean equals(Object element)	It matches two collections.
public int hashCode()	It returns the hash code number of the collection.

Iterator interface

- Iterator interface siguron iterimin e elementeve vetëm në një drejtim (drejtimin përpara).

- Ekzistojnë vetëm tre metoda në këtë interface. Ato janë:

Method	Description
public boolean hasNext()	It returns true if the iterator has more elements otherwise it returns false.
public Object next()	It returns the element and moves the cursor pointer to the next element.
public void remove()	It removes the last elements returned by the iterator. It is less used.

Iterable interface

- Ky interface është interface root për të gjitha klasat e collection. Interfaci i collection bën extend interface-in.

- Ky interface përmban vetëm një metodë abstrakte

  Iterator<T> iterator() Kthen iteratorin mbi elementet e tipit T.

Collection interface



- Ky interface zbatohet nga të gjitha klasat në collection framework.
- Deklaron metodat që do të ketë çdo collection.
- Me fjalë të tjera, mund të themi se interfaci i collection ndërton bazën nga e cila varet collection framework.

List interface

- List interface është interface fëmijë i Collection interface.
- List Interface implementohet nga klasat ArrayList, LinkedList, Vector dhe Stack.

```
List <data-type> list1= new ArrayList();
List <data-type> list2 = new LinkedList();
List <data-type> list3 = new Vector();
List <data-type> list4 = new Stack();
```

ArrayList



- Klasa ArrayList implementon List interface.
- Përdor një array dinamik për të ruajtur elementin kopjues të llojeve të ndryshme të të dhënave.
- Klasa ArrayList mban rendin e futjes së të dhënave (insertion order) dhe nuk është e sinkronizuar.
- Elementet e ruajtura në klasën ArrayList mund të aksesohen (Random Access Interface) rastësisht.

Shembull 1 - ArrayList




```
import java.util.*;
 public class ArrayList0{
 public static void main(String args[]){
  ArrayList<String> list=new ArrayList<String>();//Creating arraylist
        list.add("Mango");//Adding object in arraylist
        list.add("Apple");
        list.add("Banana");
        list.add("Grapes");
        //Printing the arraylist object
        System.out.println(list);
 }
}
```

Shembull 2 - ArrayList




```
import java.util.*;
public class ArrayList1{
    public static void main(String args[]){
    ArrayList<String> list=new ArrayList<String>();//Krijimi i arraylist
                      list.add("Test1");// shtimi i nje objekti ne arraylist
                      list.add("Test2");
                      list.add("Test1");
                      list.add("Test3");
        //Përshkrimi i listës duke përdorur Iterator
        Iterator itr=list.iterator();
                while(itr.hasNext()){
                System.out.println(itr.next());
                }
    }
}
```

Shembull 3 - ArrayList




```
import java.util.*;
    public class ArrayList3{
         public static void main(String args[]){
          ArrayList<String> list=new ArrayList<String>();//Creating arraylist
              list.add("Mango");//Adding object in arraylist
              list.add("Apple");
              list.add("Banana");
              list.add("Grapes");
          //Traversing list through for-each loop
          for(String fruit:list)
            System.out.println(fruit);

         }
    }
```

LinkedList



- LinkedList implementon Collection interface.
- Mund të ruajë elementet e kopjuara (duplicate elements)
- Mban rendin e futjes së të dhënave (insertion order) dhe nuk është e sinkronizuar.
- Në LinkedList manipulimi është i shpejtë sepse nuk kërkohet zhvendosja e elementeve

Shembull 1 - LinkedList



```java
import java.util.*;
 public class LinkedList0{
 public static void main(String args[]){
  LinkedList<String> list=new LinkedList<String>();//Creating LinkedList
        list.add("Mango");//Adding object in LinkedList
        list.add("Apple");
        list.add("Banana");
        list.add("Grapes");
        //Printing the LinkedList
        System.out.println(list);
 }
}
```

Shembull 2 - LinkedList




```
import java.util.*;
public class LinkedList1{
    public static void main(String args[]){
            LinkedList<String> al=new LinkedList<String>();
            al.add("Test1");
            al.add("Test2");
            al.add("Test1");
            al.add("Test3");
        Iterator<String> itr=al.iterator();
            while(itr.hasNext()){
            System.out.println(itr.next());
        }
    }
}
```

Shembull 3 - LinkedList




```
import java.util.*;
    public class LinkedList3{
          public static void main(String args[]){
           LinkedList<String> list=new LinkedList<String>();//Creating LinkedList
                list.add("Mango");//Adding object in LinkedList
                list.add("Apple");
                list.add("Banana");
                list.add("Grapes");
           //Traversing list through for-each loop
           for(String fruit:list)
             System.out.println(fruit);

          }
    }
```

Vector

- Vector përdorë një array dinamik për të ruajtur të dhënat.
- Është e ngjajshme me ArrayList, mirëpo është e sinkronizuar dhe përmban disa metoda të cilat nuk janë pjesë e Collection Framework

Shembull 1 - Vector



```java
import java.util.*;
public class Vector0 {
        public static void main(String args[]) {
            //Create a vector
            Vector<String> vec = new Vector<String>();
            //Adding elements using add() method of List
            vec.add("Tiger");
            vec.add("Lion");
            vec.add("Dog");
            vec.add("Elephant");

            System.out.println("Elements are: "+vec);
        }
}
```

Shembull 2 - Vector




```
import java.util.*;
    public class Vector1{
        public static void main(String args[]){
            Vector<String> v=new Vector<String>();
                v.add("Test");
                v.add("Test1");
                v.add("Test1");
                v.add("Test2");
            Iterator<String> itr=v.iterator();
                while(itr.hasNext()){
                    System.out.println(itr.next());
                }
        }
    }
```

Shembull 3 - Vector



```java
import java.util.*;
        public class Vector3{
              public static void main(String args[]){
               Vector<String> list=new Vector<String>();//Creating Vector
                    list.add("Mango");//Adding object in Vector
                    list.add("Apple");
                    list.add("Banana");
                    list.add("Grapes");
               //Traversing list through for-each loop
               for(String fruit:list)
                 System.out.println(fruit);

              }
        }
```

Hashtable

Pikat për të mbajtur mend

- Një Hashtable është array e një liste. Çdo listë është e njohur si bucket. Pozicioni i bucket identifikohet duke thirrur metodën hashcode (). Një Hashtable përmban vlera të bazuara në çelës.
- Klasa Hashtable përmban elemente unike.
- Klasa Hashtable nuk lejon vlera null (çelësin ose vlerën null).

Metodat e Hashtable

Metoda	Përshkrimi	Input	Output
Hashtable	Krijon një hashtable të re të zbrazët	Asnjë	Asnjë
put	Shton qiftin e dhënë <çelësi, vlera> në tabelë	Dy parametrat të tipit Object	Një referencë të tipit Object
get	Kthen vlerën e asocuar me çelësin e dhënë	Një parametër të tipit Object	Një referencë të tipit Object
remove	Fshinë çelësin dhe vlerën e asocuar me të nga hashtable	Një parametër të tipit Object	Një referencë të tipit Object
size	Kthen numrin e çelësave në hashtable	Asnjë	Një vlerë të tipit int

Map Interface



- Map është një strukturë e të dhënave e cila mbështetet lidhjen key-value
- Ky interface nuk mbështet çelësat e kopjuar (duplicate keys)
- Një Map është e dobishme nëse ekzistojnë të dhëna dhe ne dëshirojmë të kryejmë operacione në bazë të çelësit.

Shembull 1 - Hashtable




```
import java.util.*;
public class Hashtable0{
    public static void main(String args[]){
        Hashtable<Integer,String> hm=new Hashtable<Integer,String>();

                hm.put(1,"Test 1");
                hm.put(2,"Test 2");
                hm.put(3,"Test 3");
                hm.put(4,"Test 4");

            System.out.println(hm);
    }
}
```

Shembull 2 - Hashtable




```java
import java.util.*;
public class Book {
    int id;
    String name,author,publisher;
    int quantity;
    public Book(int id, String name, String author, String publisher, int quantity) {
        this.id = id;
        this.name = name;
        this.author = author;
        this.publisher = publisher;
        this.quantity = quantity;
    }
}
```

Shembull 2 - Hashtable




```
import java.util.*;
public class HashtableExample {
    public static void main(String[] args) {
    //Creating map of Books
        Map<Integer,Book> map=new Hashtable<Integer,Book>();
        //Creating Books
        Book b1=new Book(101,"Let us C","Yashwant Kanetkar","BPB",8);
        Book b2=new Book(102,"Data Communications & Networking","Forouzan","Mc Graw Hill",4);
        Book b3=new Book(103,"Operating System","Galvin","Wiley",6);
        //Adding Books to map
        map.put(1,b1);
        map.put(2,b2);
        map.put(3,b3);
        //Traversing map
        for(Map.Entry<Integer, Book> entry:map.entrySet()){
            int key=entry.getKey();
            Book b=entry.getValue();
            System.out.println(key+" Details:");
            System.out.println(b.id+" "+b.name+" "+b.author+" "+b.publisher+" "+b.quantity);
        }
    }
}
```